

Basi di Dati

Nicolò Giuliani

Indice

1	Introduzione	2
1.1	Sistema informativo e sistema informatico	2
1.2	Informazione e dati	2
1.3	DBMS	2
1.4	Transazioni	3
1.5	Evoluzione storica	4
1.6	DBMS e file system	4
1.7	Modelli dei dati	4
1.8	Architettura di un DBMS	5
1.9	Linguaggi per basi di dati	6
1.10	Attori e utenti	6
1.11	Pregi e difetti dei DBMS	7
2	Il modello relazionale	7
2.1	Modelli logici	7
2.2	Il modello relazionale	7
2.3	Relazione logica	8
2.4	Struttura non posizionale	8
2.5	Modello value based	8
2.6	Schemi e istanze	9
2.7	Casi particolari	9
2.8	Informazione parziale	9
2.9	Vincoli di integrità	10
2.10	Vincoli di tupla	10
2.11	Chiavi	11
2.12	Chiavi e valori nulli	12
2.13	Integrità referenziale	12
2.14	Azioni compensative	12

1 Introduzione

1.1 Sistema informativo e sistema informatico

Base di dati (significato generico): insieme organizzato di dati che supporta lo svolgimento delle attività specifiche di un ente (istituzione, azienda, ufficio, persona).

Base di dati (significato specifico): insieme di dati gestito da un DBMS.

Sistema informativo

componente di un ente che gestisce le informazioni di interesse. Ogni ente ha il proprio sistema informativo, anche se non esplicitamente identificabile nella sua organizzazione; va studiato nel suo contesto operativo. È un concetto *indipendente* dall'automazione: esistono da secoli organizzazioni la cui unica missione è gestire dati (anagrafi, banche).

Sistema informatico

la porzione automatizzata del sistema informativo, cioè la componente che gestisce le informazioni con tecnologie informatiche.

Gerarchia di inclusione: Azienda $\supset$ Organizzazione $\supset$ Sistema informativo $\supset$ Sistema informatico.

Gestione dell'informazione: raccolta/acquisizione, archiviazione/conservazione, elaborazione/trasformazione/produzione, distribuzione/comunicazione/scambio.

1.2 Informazione e dati

Le attività umane rappresentano l'informazione in molti modi (idee informali, linguaggio naturale, disegni e diagrammi, numeri e codici) su supporti diversi (mente umana, carta, dispositivi elettronici). Nei sistemi informatici invece l'informazione è rappresentata in modo povero ma essenziale: come **dati**.

- *informazione*: fatti forniti o appresi su qualcosa o qualcuno.
- *dato*: elemento di informazione, punto di partenza fissato di un procedimento.

Il dato è il risultato del processo di organizzazione, codifica e memorizzazione dell'informazione. **Il dato da solo è inutile senza la sua interpretazione** (es. i numeri su un cartello stradale non dicono nulla se non si sa che sono orari, e quale orario vale in quale giorno).

Perché i dati? È difficile rappresentare con precisione informazione e conoscenza ricche. I dati sono una *risorsa strategica* perché sono più stabili delle altre componenti (processi aziendali, tecnologie, ruoli): es. i dati di banche e anagrafi.

1.3 DBMS

DBMS (DataBase Management System): sistema che gestisce collezioni di dati

- **grandi**
- **persistenti**
- **condivise**

garantendo

- **privatezza**
- **affidabilità**
- **efficienza**
- **efficacia**

Esempi: DB2, Oracle, SQLServer, MySQL, PostgreSQL, Access, BigQuery, SQLite.

Grandi dimensioni molto maggiori della memoria centrale; il solo limite considerato è quello fisico dei dispositivi. Ordini di grandezza: 1–5 TB (dati transazionali), 30–50 TB (dati decisionali), 500–800 TB (dati scientifici), 10^{11} record.

Persistenti il loro tempo di vita è indipendente da quello dei processi che li usano.

Condivise ogni (grande) azienda è divisa in settori, ognuno con un proprio sotto-sistema informativo. Senza condivisione nascono **ridondanza** (stessi dati ripetuti) e quindi possibile **incoerenza** (descrizioni diverse non coincidono). La base di dati è una risorsa *integrata e condivisa*, da cui la necessità di **meccanismi di autorizzazione** (attività diverse sugli stessi dati) e **controllo della concorrenza** (più utenti sugli stessi dati).

Privatezza si definiscono autorizzazioni del tipo “l’utente A può leggere tutti i dati e modificare il dato X”.

Affidabilità resistenza a malfunzionamenti hardware e software. La base di dati è una risorsa preziosa e va conservata a lungo termine. Tecnica fondamentale: la gestione delle **transazioni**.

Efficienza uso proficuo delle risorse di memoria (primaria e secondaria) e di tempo (esecuzione e risposta). Poiché i DBMS offrono moltissime funzioni, potrebbero risultare inefficienti: da qui grandi investimenti e concorrenza fra prodotti. L’efficienza è anche frutto della qualità del software.

Efficacia rendere produttive le attività degli utenti, offrendo funzionalità potenti e flessibili.

1.4 Transazioni

Una **transazione** è un insieme di operazioni non separabili (*atomiche*), corrette anche in presenza di concorrenza, con effetti definitivi.

Atomicità

o si esegue tutta la sequenza o non se ne esegue nulla. Es.: “sposta denaro dal conto A al conto B: o si preleva da A e si versa su B, o non si fa nulla”.

Concorrenza

transazioni concorrenti devono restare coerenti: due assegni sullo stesso conto incassati contemporaneamente non devono perdersi; due agenzie non devono prenotare due volte lo stesso posto sul treno.

Durabilità

il **commit** garantisce che il risultato finale sia registrato e conservato, anche in caso di concorrenza o guasti.

Due punti di vista sulle transazioni:

- *utente*: un programma che realizza una funzione utile, frequente e predefinita, con poche eccezioni previste (versamento in conto, emissione di un certificato di nascita, iscrizione all'anagrafe, prenotazione di un volo). Sono scritte in un linguaggio ospite, noto o *ad hoc*.
- *sistema*: sequenza indivisibile di operazioni (cfr. affidabilità).

1.5 Evoluzione storica

Anni '70 — nessun DBMS

le applicazioni accedono direttamente ai file del sistema operativo; i dati comuni vengono duplicati nelle applicazioni.

Anni '80 — primi DBMS

fra le applicazioni (Pascal, C) e i file si interpone il DBMS, che offre tabelle.

Anni '90 — DBMS attivi

il comportamento procedurale comune viene condiviso fra le applicazioni. Nascono le **stored procedures**, che però non sono standardizzate e soffrono di *impedance mismatch* con il linguaggio usato per esprimerle; si introducono quindi i **trigger**, regole che modellano il comportamento procedurale condiviso e sono gestite dal DBMS stesso.

Anni 2000

architettura client/server su Internet: front-end (Javascript) sul client, Web Application Server e back-end (Java), database attivo con tabelle e trigger sul server.

Anni 2010

app mobili: front-end in Java, Objective-C, HTML5; il resto dell'architettura resta invariato.

1.6 DBMS e file system

Anche un file system può gestire grandi collezioni di dati, ma offre un accesso a grana grossa: “tutto o niente”. I DBMS estendono le funzionalità del file system, offrendo più servizi in modo integrato.

Descrizione dei dati ogni programma che non usa un DBMS contiene al proprio interno la descrizione della struttura dei file che tratta: programmi diversi possono avere descrizioni diverse, generando incoerenza fra le rappresentazioni. Nei DBMS una porzione della base di dati (il **catalogo** o *dizionario*) contiene una descrizione centralizzata dei dati, condivisa da tutte le applicazioni.

1.7 Modelli dei dati

Nei DBMS i dati sono descritti a diversi **livelli di astrazione**: i programmi usano una rappresentazione di alto livello indipendente da quella di livello più basso, così una modifica al livello basso non richiede di modificare i programmi. Questo è ottenuto attraverso il concetto di **modello dei dati**.

Modello dei dati: insieme di costrutti utilizzati per organizzare i dati e descriverne il comportamento. Componente fondamentale: dare struttura ai dati tramite **costruttori di tipo**. Es.: il **modello relazionale** fornisce il costruttore *relazione*, che permette di definire insiemi di record omogenei.

Schema e istanza in ogni base di dati esistono

- lo **schema**: invariante nel tempo, descrive la struttura dei dati (aspetto *intensionale*) — es. l'intestazione delle tabelle;
- l'**istanza**: i valori effettivi, che possono cambiare rapidamente (aspetto *estensionale*) — es. il corpo delle tabelle.

Due tipologie di modelli

Modelli logici

usati dai DBMS per organizzare e memorizzare i dati; usati dai programmi, indipendenti dalla rappresentazione fisica. Es.: relazionale, reticolare, gerarchico, a oggetti, XML.

Modelli concettuali

rappresentano i dati indipendentemente da ogni sistema specifico; descrivono concetti del mondo reale e si usano nella fase preliminare di progettazione. Il più diffuso è **Entity-Relationship**.

1.8 Architettura di un DBMS

Architettura semplificata (a due livelli): utente $\rightarrow$ *schema logico* $\rightarrow$ *schema interno* $\rightarrow$ DB.

Schema logico

descrive la struttura della base di dati tramite il modello logico dei dati (es. la struttura delle tabelle).

Schema interno (o fisico)

rappresenta lo schema logico a livello di memorizzazione, con strutture dati *raw* (file, record, puntatori).

Poiché la rappresentazione logica è indipendente da quella fisica — una tabella si usa “così com'è”, indipendentemente dalla rappresentazione sottostante, che può anche variare nel tempo — nel corso si considera solo il livello logico.

Architettura standard ANSI/SPARC a tre livelli aggiunge sopra lo schema logico uno o più **schemi esterni**, ciascuno usato da un gruppo di utenti:

- **schema logico**: struttura della base di dati secondo il modello logico “principale”;
- **schema interno**: rappresentazione a livello di memorizzazione;
- **schema esterno**: descrive una parte della base di dati in un modello logico (*viste* parziali, tabelle derivate, anche da modelli diversi).

Indipendenza dei dati è conseguenza della stratificazione; l'accesso ai dati avviene sempre a livello esterno (che può coincidere con quello logico).

Indipendenza fisica

livello logico ed esterno sono indipendenti dal livello fisico: una relazione è usata allo stesso modo qualunque sia la sua realizzazione fisica, che può cambiare senza modificare i programmi.

Indipendenza logica

il livello esterno è indipendente da quello logico: si può operare su una vista senza modificare il livello logico, e modifiche “trasparenti” allo schema logico lasciano inalterato lo schema esterno.

1.9 Linguaggi per basi di dati

Un altro aspetto dell'efficacia è la disponibilità di linguaggi e interfacce diverse:

- linguaggi testuali *interattivi* (**SQL**);
- comandi SQL immersi in un linguaggio **ospite** (Pascal, Java, C);
- comandi SQL immersi in un linguaggio *ad hoc* con altre funzionalità (grafici, stampa di tabelle) — es. Oracle PL/SQL;
- interfacce amichevoli, senza linguaggio testuale.

Esempio SQL interattivo date le tabelle `Corsi(Corso, Docente, Aula)` e `Aule(Nome, Edificio, Piano)`, per ottenere i corsi che si tengono al piano terra:

```
SELECT Corso, Aula, Piano
FROM Aule, Corsi
WHERE Nome = Aula
      AND Piano = 'Terra'
```

Separazione fra dati e programmi

DML

data manipulation language: interrogazione e aggiornamento delle *istanze* della base di dati.

DDL

data definition language: definizione degli *schemi* (logico, esterno, fisico) e altre operazioni. Es.:

```
CREATE TABLE orario (
  corso      CHAR(20),
  docente    CHAR(20),
  aula       CHAR(4),
  ora        CHAR(5) )
```

1.10 Attori e utenti

- progettisti e realizzatori di DBMS;
- progettisti della base di dati e amministratori (**DBA**);
- progettisti e programmatori di applicazioni;
- utenti:
 - *utenti finali* (terminalisti): eseguono operazioni predefinite (transazioni);
 - *utenti casuali*: eseguono operazioni non previste a priori, usando linguaggi interattivi.

DBA persona o gruppo responsabile del controllo e della gestione centralizzata della base di dati (efficienza, affidabilità, autorizzazioni). Spesso il DBA si occupa anche della progettazione, tranne che nei progetti complessi.

1.11 Pregi e difetti dei DBMS

Pregi

- dati come risorsa comune, base di dati come modello della realtà;
- gestione dei dati centralizzata, standardizzabile ed economia di scala;
- disponibilità di servizi integrati;
- riduzione di ridondanze e incoerenze;
- indipendenza dei dati (favorisce sviluppo e manutenzione delle applicazioni).

Difetti

- costo dei prodotti e della loro adozione;
- funzionalità non separabili (con conseguente riduzione di efficienza).

2 Il modello relazionale

2.1 Modelli logici

Esistono tre modelli logici *tradizionali*:

- **gerarchico**;
- **reticolare** (network);
- **relazionale**.

Più recenti sono il modello a *oggetti* (poco diffuso) e i modelli su *XML* (considerati complementari al relazionale).

Caratteristica distintiva

Gerarchico e reticolare

usano **riferimenti espliciti** (puntatori) fra i record.

Relazionale

è **value based**: i riferimenti fra dati memorizzati in relazioni diverse sono rappresentati tramite *valori* (non puntatori).

2.2 Il modello relazionale

Definito da **E. F. Codd** nel 1970 per garantire l'indipendenza dei dati. Implementato in DBMS reali solo a partire dal 1981: realizzare indipendenza dei dati in modo efficiente e affidabile non è banale. Il modello si basa sulla definizione logica di *relazione* (con alcune differenze) e le relazioni sono rappresentate tramite **tabelle**.

Relazione, relation, relationship tre concetti da tenere distinti:

- *relazione logica*: nel senso della teoria degli insiemi;
- *relazione* nel modello relazionale;
- *relationship*: esprime una specifica classe di fatti nel modello Entity-Relationship.

2.3 Relazione logica

Dati n insiemi $D_1, \dots, D_n$ (non necessariamente distinti):

- il **prodotto cartesiano** $D_1 \times \dots \times D_n$ è l'insieme di tutte le n -uple $(d_1, \dots, d_n)$ tali che $d_1 \in D_1, \dots, d_n \in D_n$;
- una **relazione logica** su $D_1, \dots, D_n$ è un *sottoinsieme* $r \subseteq D_1 \times \dots \times D_n$;
- i D_i sono i **domini** della relazione.

Proprietà

- è un *insieme* di n -uple: non c'è ordine fra le tuple;
- le tuple sono tutte *distinte*;
- ogni n -upla è *ordinata*: l' i -esimo valore proviene dall' i -esimo dominio.

Struttura posizionale in `Matches` $\subseteq \text{string} \times \text{string} \times \text{int} \times \text{int}$ lo stesso dominio compare con **ruoli** diversi, distinti solo dalla *posizione* (1^a colonna = squadra di casa, 2^a = ospite, ecc.). Questa è una struttura **posizionale**.

2.4 Struttura non posizionale

Nel modello relazionale a ciascun dominio si associa un **attributo**, un nome unico che ne esprime il “ruolo”. La posizione delle colonne diventa irrilevante: la struttura è **non posizionale**. Es. (`Home`, `Away`, `GoalsH`, `GoalsA`): scambiando l'ordine delle colonne (con le relative intestazioni) l'informazione non cambia; scambiare *solo* le intestazioni cambia invece il significato.

Tabelle e relazioni una tabella rappresenta una relazione se

- tutte le righe sono *diverse*;
- tutte le intestazioni di colonna sono *diverse*;
- i valori all'interno di ogni colonna sono *omogenei* (stesso dominio).

Righe e colonne possono assumere qualunque posizione.

2.5 Modello value based

I riferimenti fra dati in relazioni diverse sono espressi tramite **valori** nelle tuple (es. `Exams.Student` = `Students.Number`), non tramite puntatori. Altri modelli (reticolare, gerarchico, a oggetti) usano invece riferimenti espliciti gestiti dal sistema.

Pregi del modello value based

- indipendenza dalla struttura fisica, che può cambiare dinamicamente (una cosa analoga si può ottenere con puntatori “di alto livello”);
- si memorizza solo ciò che è rilevante dal punto di vista applicativo;
- utente e programmatore vedono gli stessi dati;
- i dati possono essere facilmente condivisi fra ambienti diversi;
- i puntatori sono direzionali, i valori no.

2.6 Schemi e istanze

Schema di una relazione un nome di relazione R con un insieme di attributi $A_1, \dots, A_n$:

$$R(A_1, \dots, A_n)$$

Es. STUDENTS(Number, Surname, Name, Year of Birth).

Schema di database relazionale insieme di schemi di relazione:

$$\mathcal{R} = \{R_1(X_1), \dots, R_k(X_k)\}$$

Tupla una tupla su un insieme di attributi X è un **mapping** che associa a ogni attributo $A \in X$ un valore nel dominio di A . Con $t[A]$ si indica il valore della tupla t sull'attributo A (es. $t[\text{Name}] = \text{Mario}$).

Istanze

- una **istanza di relazione** r è un insieme finito di tuple con schema $R(X)$;
- una **istanza di database relazionale** su $\mathcal{R} = \{R_1(X_1), \dots, R_n(X_n)\}$ è un insieme $r = \{r_1, \dots, r_n\}$ di istanze di relazione (r_i istanza di R_i).

2.7 Casi particolari

Relazioni con un solo attributo sono ammesse: es. WorkerStudents(Number), che elenca soltanto le matricole degli studenti lavoratori.

Strutture annidate molti dati reali (es. scontrini, fatture) sono naturalmente *annidati*: un'intestazione con vari campi ripetuti (righe di prodotto). Il modello relazionale è *piatto*: le strutture annidate si rappresentano con più relazioni collegate da valori. Es. Receipts(Number, Date, Total) e Courses(Receipt, Row, No, Description, Price). La rappresentazione scelta dipende dagli obiettivi: righe ripetute ammesse? ordine delle righe significativo? esistono più rappresentazioni possibili.

2.8 Informazione parziale

Il modello relazionale ha una struttura *rigida*: l'informazione è espressa in tuple che devono rispettare uno schema. Ma i dati reali possono non combaciare con il formato atteso (es. un "secondo nome" non sempre esistente).

Perché non usare valori speciali scegliere un valore del dominio (0, stringa vuota, "99", ...) per marcare l'assenza è pericoloso:

- potrebbero non esserci valori "inutilizzati" disponibili;
- quei valori potrebbero in seguito diventare significativi;
- bisognerebbe comunque tracciare a livello di software questi valori "non rappresentabili".

Valore NULL tecnica grezza ma efficace: introdurre un valore speciale **NULL** che indica l'assenza di un valore atteso e non appartiene al dominio. Formalmente, per ogni attributo A , $t[A]$ è o un valore in $\text{dom}(A)$ oppure **NULL**. Servono vincoli specifici per stabilire quando **NULL** non è ammesso.

Tipi di NULL ci sono almeno tre casi diversi di valore mancante:

- valore *sconosciuto* (esiste ma non lo conosciamo);
- valore *inesistente* (non esiste per quella tupla);
- valore *non informativo* (non sappiamo se esiste).

I DBMS moderni non distinguono fra questi casi.

Troppi NULL una istanza con troppi NULL (specialmente nelle chiavi o nei riferimenti) perde valore informativo: l'uso di NULL va limitato.

2.9 Vincoli di integrità

Alcune istanze sintatticamente corrette possono non rappresentare informazione corretta per l'applicazione (es. un voto "32", una *laude* con voto < 30 , un riferimento a uno studente inesistente).

Vincolo di integrità proprietà che deve valere sulle istanze per rappresentare informazione corretta. Formalmente è una funzione booleana (**predicato**) che associa a ogni istanza *true* o *false*.

Perché

- descrizione più accurata del dominio reale;
- supporto alla *data quality*;
- utili in fase di progettazione;
- usati nella valutazione delle interrogazioni.

Osservazione i DBMS non supportano tutti i tipi di vincolo: si dichiarano quelli supportati e il DBMS ne impedisce la violazione; per gli altri il programmatore deve implementarli fuori dal DBMS.

Classificazione

Intra-relazionali

coinvolgono una sola relazione:

- su valori (*domain constraints*);
- su tuple.

Inter-relazionali

coinvolgono più relazioni.

2.10 Vincoli di tupla

Esprimono regole sui valori di ogni singola tupla, indipendentemente dalle altre. Caso specifico: i **domain constraints** coinvolgono un solo attributo.

Sintassi espressione booleana di atomi che confrontano:

- valori del dominio degli attributi, es. $(\text{Grade} \geq 18) \text{ AND } (\text{Grade} \leq 30)$;
- espressioni aritmetiche su tali valori, es. $\text{GrossPay} = (\text{Deductions} + \text{Net})$.

Una tupla (Bruni, 50.000, 11.000, 36.000) viola il vincolo perché $11.000 + 36.000 \neq 50.000$.

2.11 Chiavi

Nel mondo reale ogni entità è identificabile: nella tabella **Students** ogni tupla ha un **Number** unico e non esistono tuple con stessi **Surname**, **Name**, **Date of Birth**. Formalizziamo:

Superchiave

un insieme K di attributi di r tale che non esistono due tuple distinte $t_1, t_2 \in r$ con $t_1[K] = t_2[K]$.

Chiave

una superchiave *minimale*: nessun suo sottoinsieme proprio è già superchiave.

Esempi in **Students**(Number, Surname, Name, Curriculum, Birth):

- **Number** è chiave: è superchiave (identifica ciascuna tupla) e contiene un solo attributo, quindi è minimale;
- {Surname, Name, Birth} è un'altra chiave: è superchiave ed è minimale;
- {Surname, Curriculum} potrebbe essere superchiave in un'istanza particolare ma non nel caso generale.

Vincoli, schema e stati

- i vincoli corrispondono a proprietà del mondo reale, modellate nel DB;
- sono modellati sullo *schema* (si applicano a tutte le tuple);
- ogni schema può avere un insieme di vincoli: le tuple memorizzate sono **corrette** (valide) se soddisfano tutti i vincoli;
- un'istanza può soddisfare *per caso* altri vincoli non dichiarati.

Esistenza delle chiavi ogni relazione:

- non può contenere tuple con tutti i valori uguali (è un insieme);
- ha come superchiave l'insieme di tutti i suoi attributi;
- quindi ha almeno una chiave.

Importanza

- l'esistenza di almeno una chiave garantisce l'*accessibilità* di ogni tupla;
- le chiavi permettono di correlare tuple fra relazioni diverse: il modello relazionale è *value based*.

2.12 Chiavi e valori nulli

Se una tupla contiene NULL sugli attributi della chiave, i suoi valori non permettono di:

- identificare la tupla;
- creare facilmente riferimenti esterni da altre relazioni.

L'uso di NULL sulle chiavi va limitato.

Chiave primaria fra le chiavi di una relazione se ne sceglie una (**primary key**) che *non ammette valori NULL*. Convenzionalmente si indica con la sottolineatura degli attributi nello schema.

2.13 Integrità referenziale

Le informazioni fra relazioni diverse sono correlate tramite valori comuni, in particolare valori di chiavi (primarie). Tali correlazioni devono essere *coerenti*: es. il codice del vigile in `Infringement.Policeman` deve corrispondere a un ID esistente in `Policemen`.

Vincolo di integrità referenziale (*foreign key*) fra gli attributi X di una relazione R_1 e una relazione R_2 : i valori di X in R_1 devono comparire come valori della chiave primaria di R_2 .

Esempio in un DB con `Infringement(Code, Date, Policeman, Country, Plate)`, `Policemen(ID, Surname, Name)`, `Car(Country, Plate, Surname, Name)`:

- $\text{Policeman} \rightarrow \text{Policemen}(\text{ID})$;
- $(\text{Country}, \text{Plate}) \text{ in } \text{Infringement} \rightarrow \text{Car}(\text{Country}, \text{Plate})$.

Una tupla `Infringement` con `(Italy, AJ046EZ)`, mentre in `Car` `AJ046EZ` appartiene alla `France`, viola il vincolo.

Osservazioni

- ruolo cruciale nel modello value based;
- i vincoli possono essere *rilassati* in presenza di NULL;
- le violazioni causate da operazioni possono essere gestite tramite *azioni compensative*;
- attenzione ai vincoli definiti su più attributi: l'*ordine* degli attributi è rilevante (in `Crashes(Code, CountryA, PlateA, CountryB, PlateB)` servono due vincoli distinti verso `Car(Country, Plate)`, uno per (A, A) e uno per (B, B)).

2.14 Azioni compensative

Quando una modifica (es. cancellazione di una tupla referenziata) violerebbe un vincolo referenziale, il DBMS può reagire in modi diversi.

Comportamento standard

Restrict

rifiuta la cancellazione.

Azioni compensative possibili

Cascade

rimuove “a catena” le tuple referenzianti (es. eliminando il progetto XYZ da `Projects` si eliminano anche i dipendenti in `Employees` con `Project = XYZ`).

Set NULL

pone NULL nell'attributo referenziante (`Employees.Project` diventa NULL per chi lavorava a XYZ).

Set default

introduce un valore di default.